```
puts "ISBN   = #{book.isbn}"
puts "Price  = #{book.price}"
```

produces:

```
ISBN   = isbn1
Price  = 12.34
```

This is the first time we've used *symbols* in this chapter. As we discussed on page 24, symbols are just a convenient way of referencing a name. In this code, you can think of :isbn as meaning the *name* isbn and think of plain isbn as meaning the *value* of the variable. In this example, we named the accessor methods isbn and price. The corresponding instance variables are @isbn and @price. These accessor methods are identical to the ones we wrote by hand earlier—they will return the value of the instance variable whose name matches the name of the accessor method. These methods only allow you to read the attribute, and not to change it.

There's a common misconception that the attr_reader declaration actually declares instance variables. It doesn't. It creates the accessor methods, but the variables themselves don't need to be declared. An instance variable just pops into existence when you assign a value to it, and any instance value that hasn't been assigned a value returns nil when accessed. Ruby completely decouples instance variables and accessor methods, as we'll see in Attributes Are Just Methods Without Arguments, on page 40.

Writing to Attributes

Sometimes you need to be able to set an attribute from outside the object. For example, let's assume that we have to discount the price of some titles after reading in the raw scan data.

In other languages that restrict access to instance variables such as C# and Java, you'd do this with *setter functions*:

```
// Java code
class JavaBookInStock {
    private double _price;
    public double getPrice() {
        return _price;
    }
    public void setPrice(double newPrice) {
        _price = newPrice;
    }
}
b = new JavaBookInStock(....);
b.setPrice(calculate_discount(b.getPrice()));
```

In Ruby, the attributes of an object can be accessed via the getter method and that access looks the same as any other method. We saw this earlier with phrases such as book.isbn. So, it seems natural for setting the value of an attribute to look like normal variable assignment, such as book.isbn = "new isbn". You enable that assignment by creating a Ruby method whose name ends with an equals sign. A method so named can be used as the target of assignments:

tut_classes/stock_stats/book_in_stock_5.rb

```
class BookInStock
  attr_reader :isbn, :price

  def initialize(isbn, price)
```